C291 - Spring 25 - System Programming with C & Unix

Assignment 5

1. Generate k-grams (20 marks)

A k-gram is a k-length subsequence from a string. K-grams are used to divide a document (string/text) into fixed-length sequences which can be used to represent the document in many natural language processing tasks.

For a string of length l, there are l - k+1 k-grams.

For example if the string is “abcde” and k = 2 then generated k-grams are: ab, bc, cd, de

Write a program to generate and print the k-grams from a given string and k. You cannot use any character or string library function in this program.

Implementation Details:

Your program will define a character array and initialize it to the following string “**because the arrays starting address is passed, the called function knows precisely where the array is stored**”. To store the k-grams an n * (k+1) character array will be defined, where n is the number and k is the size of k-grams (+1 is to hold the null character). The value of k will be provided by the user.

```
/*
        Function: generateKGrams
        --------------------------------

        generate k-length grams for the input string

        k: length of k-gram
        input: input string

        returns: void
*/
void generateKGrams(int k, char input[]){
}
```

```
/*
        Function: printKGrams
        ---------------------------------

        Print all the k-length grams

        n: number of k-grams
        k: length of k-gram
        kGrams[][k+1] = generated k grams

        returns: void
*/
void printKGrams(int n, int k, char kGrams[][k+1]){
}

/*
        Function: length
        ---------------------------------
        Determine the length of a string

        str: input string to check length

        returns: length of the string
*/
int length(char str[]) {
}
```

Example 1:
String: **because the arrays starting address is passed, the called function knows precisely where the array is stored**
Input value of k: **4**

Output (You don't have to print the same output. You can print in any way, but it should be clear):

['beca', 'ecau', 'caus', 'ause', 'use ', 'se t', 'e th', ' the', 'the ', 'he a', 'e ar', ' arr', 'arra', 'rray', 'rays', 'ays ', 'ys s', 's st', ' sta', 'star', 'tart', 'arti', 'rtin', 'ting', 'ing ', 'ng a', 'g ad', ' add', 'addr', 'ddre', 'dres', 'ress', 'ess ', 'ss i', 's is', ' is ', 'is p', 's pa', ' pas', 'pass', 'asse', 'ssed', 'sed,', 'ed, ', 'd, t', ', th', ' the', 'the ', 'he c', 'e ca', ' cal', 'call', 'alle', 'lled', 'led ', 'ed f', 'd fu', ' fun', 'func', 'unct', 'ncti', 'ctio', 'tion', 'ion ', 'on k', 'n kn', ' kno', 'know', 'nows', 'ows ', 'ws p', 's pr', ' pre', 'prec', 'reci', 'ecis', 'cise', 'isel', 'sely', 'ely ', 'ly w', 'y wh', ' whe', 'wher', 'here', 'ere ', 're t', 'e th', ' the', 'the ', 'he a', 'e ar', ' arr', 'arra', 'rray', 'ray ', 'ay i', 'y is', ' is ', 'is s', 's st', ' sto', 'stor', 'tore', 'ored']

Note: The solution for this problem should be named as solution_1.c and it should be under C291-Spring-25/assignment5 where assignment5 would be the new directory you would create.

2. Student Directory (20 marks)

A high school would like to store all their student's information in one place to be able to filter it by grade, age, and year in the future. Write a program to store this data and apply a filter.

Implementation Details:

You must initialize a 2D Array which holds information about all the students that are in the high school, where each individual array holds the information of a student in the following order:

[studentlD, studentAge, studentYear, studentGrade]

You need to populate/hardcode the values for 5 students in this 2D array.

All of the following variables should be integers, studentYear should go from 9-12 and studentGrade should go from 0-100.

While the array will be hardcoded, the filtering information will be provided by the user. Prompt the user to input minimum age, minimum year and minimum grade as input.

If a **user inputs "-1"** for any of the parameters, those parameters **should not be used** for filtering.
Implement the following function

```
#define NCOLS 4

/*
        Function: filterStudents
        --------------------------------

        compare with 2D array and displays the students IDs from the students array that satisfy the
        given parameters. If no students match, print an appropriate message.

        minAge: minimum age of student to filter
        minYear: minimum Year of student to filter
        minGrade: minimum Grade of student to filter
        students: Student's information

        returns: void
*/
void filterStudents(int students[][NCOLS], int minAge, int minYear, int minGrade) {
}
```

Example 1:

Assuming the 2D array is hardcoded with the following values

1 18 11 90
2 16 10 75
3 15 9 85
4 10 10 60
5 11 9 60

Input
Enter filtering parameters:
-1 -1 80

Output
1 3

Explanation

In example we are scanning 5 student details in appropriate order of parameters.
Our student records might virtually look something like this as we store them in a 2D array:

```
students =  [[1, 18, 11, 90],
             [2, 16, 10, 75],
             [3, 15, 9, 85],
            [4, 10, 10, 60],
            [5, 11, 9, 60]
            ]
```

Now we input filter parameters and output results satisfying these filters. For given parameters we just consider 80 for minGrade and output student IDs 1 and 3 as our result(since their grades are more than 80). Observe how we ignore the filters minAge and minYear since their values were given as '-1'.

filterStudents(students, -1, -1, 80) -> 1 3

Example 2:

Assuming the 2D array is hardcoded with the following values

1 18 11 90
2 16 10 75
3 15 9 85
4 10 10 60
5 11 9 60

Input:
Enter filtering parameters:
16 -1 83

Output
1

Note: The solution for this problem should be named as solution_2.c and it should be under C291-Spring-25/assignment5 where assignment5 would be the new directory you would create.

3. Remove duplicates (20 marks)

Write a program that takes a sorted int array as input and removes duplicates if any from the array.

Implementation Details:

```
/*
        Function: printUniqueElements
        --------------------------------

        prints unique elements

        lenArray: length of the input array
        elements: input array

        returns: void
*/
void printUniqueElements(int elements[], int lenArray) {
}
```

In a sorted array, all the **duplicate elements** in the array will **appear together**.

Display the **array of unique** elements.

Example 1:
Input Size of Array : 11
Input**:** 0 0 1 1 1 2 2 3 3 4
Output**:** 0 1 2 3 4

(since input array is [0,0,1,1,1,2,2,3,3,4], and after removing duplicates we get the array as [0,1,2,3,4])

Example 2:
Input Size of Array 1: 7
Input**:** 1 2 3 4 4 5 5
Output**:** 1 2 3 4 5

Note: The solution for this problem should be named as solution_3.c and it should be under C291-Spring-25/assignment5 where assignment4 would be the new directory you would create.

4. Jaccard Similarity (20 marks)

Write a program to compute the Jaccard similarity between two sets. The Jaccard similarity of sets A and B is the ratio of the size of their intersection to the size of their union

Example:

Let say,

A = {1, 2, 5, 6}
B = {2, 4, 5, 8}

then

A ∩ B = {2, 5} and A ∪ B = {1, 2, 4, 5, 6, 8}

then

| A ∩ B | / | A ∪ B | = 2/6, so the **Jaccard similarity is 0.333**.

Implementation Details:

We will **use arrays to represent sets**,

```
/*
        Function: checkSet
        ----------------------------------

        check if the array is non empty

        input: Input array
        input_length: length of input array

        returns: void
*/
void checkSet(int input[], int input_length){
}
```

```
/*
        Function: findIntersection
        ----------------------------------

        Finds intersection between two arrays.

        input1: first input array
        input2: second input array
        input1_length: length of first input array
        input2_length: length of second input array

        returns: number of similar elements
*/
int findIntersection(int input1[], int input1_length, int input2[], int input2_length){
}

/*
        Function: findUnion
        ----------------------------------

        Finds union between two arrays.

        input1: first input array
        input2: second input array
        input1_length: length of first input array
        input2_length: length of second input array

        returns: number of distinct elements
*/
int findUnion(int input1[], int input1_length , int input2[], int input2_length){
}

/*
        Function: calculateJaccard
        ----------------------------------

        Calculate Jaccard similarity value.

        input1: first input array
        input2: second input array
        input1_length: length of first input array
        input2_length: length of second input array

        returns: Jaccardsimilarity
*/
```

```
float calculateJaccard(int input1[], int input1_length, int input2[], int input2_length)) {
}
```

Example 1:

Input**:**

Input first set length: 4
Input first set**:** 1, 2, 5, 6
Input set length 2: 4
Input second set: 2, 4, 5, 8

Output**: Jaccard similarity is 0.333**

Example 2:

Input**:**
Input first set length: 4
Input first set**:**1, 2, 5, 6
Input second set length: 2
Input second set: 8, 9
Output**: Jaccard similarity is 0**

Example 3:

Input:
Input first set length: 0
Input first set**:**
Output: set cannot be empty .

Note: The solution for this problem should be named as solution_4.c and it should be under C291-Spring-25/assignment5 where assignment5 would be the new directory you would create.

5. Merge Sorted Arrays (20 marks)

Write a program which merges 2 sorted arrays A and B into one sorted array C

Algorithm:

1. Take 2 indices i = 0 and j = 0.
2. Create an additional array C of size equal to the sum of the length of the array A and array B to store the values
3. i points to the beginning of sorted array A, j points to the beginning of sorted array B
4. Iterate until i or j reaches at the end of their respective index i.e., length of A to length of B
    a. If A[i] < B[j]
        i. Add A[i] to C
        ii. Increment i index by 1

    b. Else if A[i] > B[j]
        i. Add B[j] in C
        ii. Increment j index by 1

    c. Else // same values
        i. Add A[i] and B[j] in C
        ii. Increment value i and j by 1

5. While i < sizeA
    a. Add A[i] in C
    b. Increment i by 1

While j < sizeB

. a. Add B[j] in C
    b. Increment j by 1

Examples:

Example 1: Duplicates

Input Size of Array 1: 5

Input Array 1 : 1 2 5 7 8
Input Size of Array 2: 4
Input Array 2 : 1 3 4 6
Output : 1 1 2 3 4 5 6 7 8

Example 2: Without Duplicates
Input Size of Array 1: 4
Input Array 1 : 2 5 7 8
Input Size of Array 2: 3
Input Array 2 : 3 4 6
Output : 2 3 4 5 6 7 8

Example 3: Empty array
Input Size of Array 1:0
Input Array 1 : // empty array Input
Size of Array 2: 3
Input Array 2 : 1 2 3
Output : 1 2 3

Implementation Details:

```
/*
    Function: printSortedMergedArray
    --------------------------------
    print sorted merged int array.
    check the edge cases explained in the examples
    do not use any library functions

    array1: first input array
    array2: second input array
    lenArray1: length of first input array
    lenArray2: length of second input array

    returns: void
*/
void printSortedMergedArray(int array1[], int lenArray1, int array2[], int lenArray2) {
}
```

Note: The solution for this problem should be named as solution_5.c and it should be under C291-Spring-25/assignment5 where assignment5 would be the new directory you would create.